

17-advanced-database

Modul 17: Database Lanjutan

Tujuan Pembelajaran: Setelah mempelajari modul ini, siswa mampu:

- Memahami dan menggunakan berbagai jenis JOIN
 - Membedakan Subquery dan CTE serta kapan menggunakannya
 - Menggunakan Window Functions untuk analisis data
 - Mengelola transaksi database dengan BEGIN/COMMIT/ROLLBACK
 - Merancang strategi indexing yang efektif
 - Memahami konsep connection pooling
 - Menganalisis performa query dengan EXPLAIN ANALYZE
-

1. JOIN Lanjutan

JOIN menggabungkan baris dari dua atau lebih tabel berdasarkan kolom yang berelasi. Berikut jenis-jenis JOIN yang paling umum:

1.1 INNER JOIN

Mengembalikan **hanya baris yang cocok** di kedua tabel. Irisan dari dua himpunan.

Tabel A	Tabel B
+-----+	+-----+
X	X
X	X
+-----+	+-----+

INNER JOIN → hanya area irisan

```
SELECT siswa.nama, kelas.nama_kelas
FROM siswa
INNER JOIN kelas ON siswa.kelas_id = kelas.id;
```

1.2 LEFT JOIN (LEFT OUTER JOIN)

Mengembalikan **semua baris dari tabel kiri**, plus baris yang cocok dari tabel kanan. Jika tidak cocok, kolom kanan berisi NULL.

Tabel A	Tabel B
+-----+	+-----+
X	X
X	
X	X
+-----+	+-----+

LEFT JOIN → semua A + irisan

```
SELECT siswa.nama, pembayaran.jumlah
FROM siswa
LEFT JOIN pembayaran ON siswa.id = pembayaran.siswa_id;
```

1.3 RIGHT JOIN (RIGHT OUTER JOIN)

Kebalikan LEFT JOIN — semua baris dari **tabel kanan**, plus yang cocok dari kiri.

Tabel A	Tabel B
+-----+	+-----+
X	X
	X
X	X
+-----+	+-----+

RIGHT JOIN → semua B + irisan

```
SELECT siswa.nama, pembayaran.jumlah
FROM siswa
RIGHT JOIN pembayaran ON siswa.id = pembayaran.siswa_id;
```

1.4 FULL JOIN (FULL OUTER JOIN)

Mengembalikan **semua baris** dari kedua tabel. Data yang tidak cocok diisi NULL di sisi lawan.

Tabel A	Tabel B
+-----+	+-----+
X	X
X	
+-----+	+-----+

FULL JOIN → semua A + semua B

```
SELECT siswa.nama, pembayaran.jumlah
FROM siswa
FULL JOIN pembayaran ON siswa.id = pembayaran.siswa_id;
```

1.5 Self JOIN

JOIN tabel dengan dirinya sendiri. Berguna untuk data hierarki seperti tabel karyawan-manajer.

```
SELECT e.nama AS karyawan, m.nama AS manajer
FROM karyawan e
LEFT JOIN karyawan m ON e.manajer_id = m.id;
```

2. Subquery vs CTE (Common Table Expression)

2.1 Subquery

Subquery adalah query di dalam query. Bisa ditempatkan di SELECT, FROM, atau WHERE.

```
SELECT nama, gaji
FROM karyawan
WHERE gaji > (
    SELECT AVG(gaji) FROM karyawan
);
```

Kelebihan: Sederhana untuk kasus kecil.

Kekurangan: Sulit dibaca jika bersarang dalam; tidak bisa dipakai ulang dalam query yang sama.

2.2 CTE (Common Table Expression)

CTE membuat "tabel sementara" bernama dengan klausa WITH. Lebih rapi dan bisa dipakai berulang.

```
WITH rata_rata_gaji AS (
    SELECT AVG(gaji) AS avg_gaji FROM karyawan
)
SELECT nama, gaji
FROM karyawan, rata_rata_gaji
WHERE gaji > rata_rata_gaji.avg_gaji;
```

CTE juga memungkinkan query rekursif:

```
WITH RECURSIVE hierarki AS (
    SELECT id, nama, manajer_id, 1 AS level
    FROM karyawan WHERE manajer_id IS NULL
    UNION ALL
    SELECT k.id, k.nama, k.manajer_id, h.level + 1
    FROM karyawan k
    JOIN hierarki h ON k.manajer_id = h.id
```

```
)
SELECT * FROM hierarki;
```

2.3 Kapan Pakai Apa?

Situasi	Pilihan
Query sederhana, sekali pakai	Subquery
Perlu pakai hasil yang sama berkali-kali	CTE
Query bersarang dalam (3+ level)	CTE
Query rekursif (pohon, graph)	CTE Wajib
Subquery di SELECT/WHERE (satu nilai)	Subquery

3. Window Functions

Window functions melakukan kalkulasi **antar baris** tanpa menggabungkan baris seperti GROUP BY. Sintaks:

```
fungsi_window() OVER (
  PARTITION BY kolom1
  ORDER BY kolom2
  [ROWS ...]
)
```

3.1 ROW_NUMBER()

Memberi nomor urut unik per baris dalam partisi.

```
SELECT nama, kelas_id, nilai,
       ROW_NUMBER() OVER (PARTITION BY kelas_id ORDER BY nilai DESC) AS peringkat
FROM rapor;
```

3.2 RANK() dan DENSE_RANK()

Fungsi	Nilai Sama	Lonjakan
RANK()	Rank sama, ada lonjakan	1, 1, 3, 4
DENSE_RANK()	Rank sama, tanpa lonjakan	1, 1, 2, 3
ROW_NUMBER()	Unik semua kasus	1, 2, 3, 4

```
SELECT nama, nilai,
       RANK() OVER (ORDER BY nilai DESC) AS rank,
```

```

        DENSE_RANK() OVER (ORDER BY nilai DESC) AS dense_rank,
        ROW_NUMBER() OVER (ORDER BY nilai DESC) AS row_num
FROM rapor;

```

3.3 LAG() dan LEAD()

Mengakses baris sebelumnya (LAG) atau baris berikutnya (LEAD) dalam urutan.

```

SELECT tanggal, penjualan,
       LAG(penjualan, 1) OVER (ORDER BY tanggal) AS penjualan_sebelumnya,
       penjualan - LAG(penjualan, 1) OVER (ORDER BY tanggal) AS perubahan
FROM laporan_penjualan;

```

LEAD() untuk melihat nilai baris berikutnya:

```

SELECT tanggal, penjualan,
       LEAD(penjualan, 1) OVER (ORDER BY tanggal) AS penjualan_berikutnya
FROM laporan_penjualan;

```

3.4 Aggregate Window Functions

Fungsi agregat biasa (SUM, AVG, COUNT) bisa jadi window function:

```

SELECT tanggal, penjualan,
       SUM(penjualan) OVER (ORDER BY tanggal) AS total_kumulatif,
       AVG(penjualan) OVER (ORDER BY tanggal ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS rata_rata_7_hari
FROM laporan_penjualan;

```

4. Transaksi (BEGIN / COMMIT / ROLLBACK)

Transaksi adalah unit kerja yang **atomic** — semua berhasil atau semua gagal. Konsep ini disebut **ACID** (Atomicity, Consistency, Isolation, Durability).

4.1 Sintaks Dasar

```

BEGIN;                                -- mulai transaksi

UPDATE rekening SET saldo = saldo - 100000 WHERE id = 1;
UPDATE rekening SET saldo = saldo + 100000 WHERE id = 2;

COMMIT;                               -- simpan permanen

```

```
-- atau jika gagal:
ROLLBACK;          -- batalkan semua perubahan
```

4.2 Skenario Nyata: Transfer Bank

```
BEGIN;

UPDATE rekening SET saldo = saldo - 50000 WHERE id = 'A001';
-- jika listrik padam di sini, tanpa transaksi uang hilang

UPDATE rekening SET saldo = saldo + 50000 WHERE id = 'B002';

COMMIT;
-- hanya setelah COMMIT data benar-benar tersimpan
```

Jika ada error sebelum COMMIT, jalankan ROLLBACK untuk mengembalikan data ke keadaan awal.

4.3 Savepoint

Sub-titik di dalam transaksi untuk rollback parsial:

```
BEGIN;
INSERT INTO log VALUES ('mulai');
SAVEPOINT sp1;
INSERT INTO log VALUES ('tahap_1');
-- ups, salah
ROLLBACK TO SAVEPOINT sp1;
-- log 'tahap_1' dihapus, 'mulai' tetap ada
COMMIT;
```

4.5 Transaction Isolation Levels

Isolation level ngontrol gimana transaksi "lihat" perubahan dari transaksi lain yang jalan bersamaan. Ada 4 level di PostgreSQL:

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Use Case
READ UNCOMMITTED	❌ (PG treat as RC)	❌	❌	Jarang
READ COMMITTED (default)	✅ Aman	❌ Bisa beda	❌ Bisa beda	Web a
REPEATABLE READ	✅ Aman	❌ Aman	❌ Bisa beda	Lapora
SERIALIZABLE	✅ Aman	❌ Aman	❌ Aman	Tabung

Dirty Read: Baca data yang belum di-COMMIT. PostgreSQL ga punya ini bahkan di level terendah.

Non-Repeatable Read: Baca baris yang sama 2x, dapet hasil beda karena transaksi lain udah ngubah:

```
-- Transaksi A                                -- Transaksi B
BEGIN;                                          BEGIN;
SELECT saldo FROM rekening WHERE id=1;
-- saldo = 100rb

UPDATE rekening SET saldo=50rb WHERE
COMMIT;

SELECT saldo FROM rekening WHERE id=1;
-- saldo = 50rb (beda!)
COMMIT;
```

Phantom Read: Query filter yang sama 2x, dapet baris beda karena transaksi lain nambahin data baru.

Contoh pake REPEATABLE READ:

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

SELECT SUM(saldo) FROM rekening WHERE user_id = 1;
-- 200rb

-- Transaksi lain transfer masuk 50rb & commit
-- Transaksi ini masih liat 200rb (konsisten)

SELECT SUM(saldo) FROM rekening WHERE user_id = 1;
-- Masih 200rb – REPEATABLE READ menjamin konsistensi

COMMIT;
-- Next query baru liat data terbaru
```

Pilih level mana? | Level | Recommended buat | |-----|-----|
| | READ COMMITTED | Default. Kebanyakan web app. | | REPEATABLE
READ | Laporan, histori, billing — butuh snapshot konsisten | | SERI-
ALIZABLE | Keuangan, antrian, race condition sensitif — tapi siap-siap
retry |

⚠ **Makin tinggi isolation, makin banyak conflict/rollback.** SERIALIZABLE bisa nge-drop transaksi kalo detect konflik — harus pake retry logic di aplikasi.

4.6 Transaksi di Aplikasi (Node.js)

Pake library pg — transaksi manual:

```
import { Pool } from 'pg';
const pool = new Pool({ connectionString: process.env.DATABASE_URL });
```

```

async function transferMoney(fromId: number, toId: number, amount: number) {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');

    await client.query(
      'UPDATE rekening SET saldo = saldo - $1 WHERE id = $2',
      [amount, fromId]
    );

    // Kalo error di sini, ROLLBACK otomatis
    await client.query(
      'UPDATE rekening SET saldo = saldo + $1 WHERE id = $2',
      [amount, toId]
    );

    await client.query('COMMIT');
    console.log('Transfer berhasil');
  } catch (err) {
    await client.query('ROLLBACK');
    console.error('Transfer gagal, di-rollback:', err);
    throw err; // Lempar ke caller buat di-handle
  } finally {
    client.release(); // Kembalikan koneksi ke pool
  }
}

```

Pattern: Transaction with retry on serialization error:

```

async function executeWithRetry<T>(
  fn: (client: PoolClient) => Promise<T>,
  maxRetries = 3
): Promise<T> {
  const client = await pool.connect();
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      await client.query('BEGIN');
      const result = await fn(client);
      await client.query('COMMIT');
      return result;
    } catch (err: any) {
      await client.query('ROLLBACK');
      // Cuma retry kalo error serialization (40001)
      if (err.code === '40001' && attempt < maxRetries) {
        const backoff = Math.min(100 * Math.pow(2, attempt - 1), 2000);
        await new Promise(r => setTimeout(r, backoff));
      }
    }
  }
}

```



```

        continue;
    }
    throw err;
} finally {
    client.release();
}
}
throw new Error('Gagal setelah retry max');
}

// Pake:
await executeWithRetry(async (client) => {
    await client.query('UPDATE ...');
    await client.query('UPDATE ...');
});

```

4.4 Best Practices Transaksi

- **Transaksi pendek.** Jangan input user atau API call di dalam transaksi.
- **Error handling wajib.** Setiap BEGIN harus ada COMMIT/ROLLBACK — pake try/catch/finally.
- **Test isolation level.** Paling rendah yang cukup — jangan SERIALIZABLE kalo ga perlu.
- **Retry serialization failure.** Kode 40001 — transaksi gagal karena konflik, tinggal ulang.
- **Jangan nested transaction.** PostgreSQL ga punya nested transaction — pake SAVEPOINT aja.

5. Strategi Indexing

Index mempercepat pencarian dengan cara seperti **kamus** — daripada membaca seluruh buku, langsung ke halaman kata yang dicari.

5.1 B-Tree Index

Jenis index **default** di PostgreSQL. Cocok untuk:

- Operator =, <, >, <=, >=, BETWEEN
- ORDER BY dan GROUP BY
- LIKE 'prefix%' (bukan '%suffix')

```
CREATE INDEX idx_siswa_nama ON siswa (nama);
```

Struktur B-Tree seimbang: semua leaf node di level yang sama, jadi waktu pencarian **$O(\log n)$** .

5.2 Composite Index (Multi-Kolom)

Index pada lebih dari satu kolom. Urutan kolom **sangat penting**.

```
CREATE INDEX idx_kelas_tanggal ON pembayaran (kelas_id, tanggal_bayar);
```

Aturan praktis: Kolom yang paling sering difilter diletakkan di depan.

Query ini bisa memanfaatkan index di atas:

```
SELECT * FROM pembayaran
WHERE kelas_id = 3 AND tanggal_bayar >= '2025-01-01';
```

Query ini **tidak** memanfaatkan index di atas:

```
SELECT * FROM pembayaran
WHERE tanggal_bayar >= '2025-01-01';
-- kolom pertama kelas_id tidak disebut, index tidak efektif
```

5.3 Kapan TIDAK Perlu Index

Situasi	Alasan
Tabel kecil (< 1000 baris)	Full scan lebih cepat dari index lookup
Kolom dengan sedikit nilai unik (boolean, status)	Index tidak selektif
Kolom sering di-INSERT/UPDATE/DELETE	Index memperlambat write
Query LIKE '%kata%' (wildcard depan)	Index tidak bisa dipakai
Tabel jarang di-query	Biaya maintenance tidak sebanding

5.4 Melihat Penggunaan Index

```
-- PostgreSQL: cek apakah index dipakai
EXPLAIN ANALYZE SELECT * FROM siswa WHERE nama = 'Budi';
```

Jika terlihat Seq Scan → index tidak dipakai. Jika Index Scan → index bekerja.

6. Connection Pooling

6.1 Kenapa Diperlukan?

Setiap koneksi database makan **resource**: RAM, CPU, file descriptor. Jika aplikasi web melayani 1000 request per detik dan setiap request

buka-tutup koneksi, database akan kewalahan.

Masalah tanpa pooling:

Request 1 → buka koneksi → query → tutup koneksi

Request 2 → buka koneksi → query → tutup koneksi

□

Request N → buka koneksi → query → tutup koneksi

Biaya buka-tutup koneksi tinggi (TCP handshake, autentikasi, alokasi memori). Database juga punya batas maksimal koneksi (default PostgreSQL: 100).

6.2 Solusi: Connection Pool

Pool menyimpan sejumlah **koneksi tetap** yang dipakai ulang. Aplikasi meminjam koneksi dari pool, bukan membuat baru.

Aplikasi → Connection Pool (10 koneksi) → Database
 ↑ pinjam ↑ pinjam ↑ pinjam
 Request 1 Request 2 Request 3

Keuntungan:

- Beban database stabil (koneksi konstan)
- Latensi turun drastis (tidak perlu handshake tiap kali)
- Resource terkelola (tidak boros)

6.3 pgBouncer

pgBouncer adalah connection pooler khusus untuk PostgreSQL. Ringan (~1 MB RAM), cepat, dan sangat stabil.

instalasi di Ubuntu/Debian

`sudo apt install pgbouncer`

konfigurasi: /etc/pgbouncer/pgbouncer.ini

[databases]

db_rpl = host=127.0.0.1 port=5432 dbname=db_rpl

[pgbouncer]

listen_addr = 127.0.0.1

listen_port = 6432

pool_mode = transaction *# mode paling umum*

max_client_conn = 500

default_pool_size = 20

Aplikasi kemudian tersambung ke **port 6432** (pgBouncer) bukan **5432** (PostgreSQL langsung).

Mode Pooling:

Mode	Koneksi Dilepas Setelah	Cocok Untuk
session	Koneksi ditutup	Aplikasi lama / stateful
transaction	Transaksi selesai	Web apps (paling umum)
statement	Query selesai	Hati-hati — risiko error

6.4 Pooling di Aplikasi

Python (psycopg2):

```
from psycopg2 import pool

connection_pool = pool.ThreadedConnectionPool(
    minconn=2, maxconn=10,
    host="localhost", port="5432", # pgBouncer
    dbname="db_rpl", user="user", password="pass"
)

conn = connection_pool.getconn() # pinjam
cur = conn.cursor()
cur.execute("SELECT * FROM siswa")
connection_pool.putconn(conn)    # kembalikan
```

7. Performa Query dengan EXPLAIN ANALYZE

7.1 Cara Membaca

```
EXPLAIN ANALYZE SELECT * FROM siswa WHERE kelas_id = 3;
```

Output menunjukkan **rencana eksekusi** dan **waktu aktual**:

```
Seq Scan on siswa (cost=0.00..35.50 rows=10 width=512)
    (actual time=0.015..0.045 rows=10 loops=1)
    Filter: (kelas_id = 3)
    Rows Removed by Filter: 500
Planning Time: 0.080 ms
Execution Time: 0.055 ms
```

7.2 Istilah Penting

Istilah	Arti
Seq Scan	Full table scan — baca semua baris (lambat untuk tabel besar)
Index Scan	Scan via index (cepat)
Index Only Scan	Semua data sudah di index, tidak perlu baca tabel
cost	Perkiraan biaya (units) — first..total
actual time	Waktu real dalam ms — start..end
rows	Perkiraan vs actual jumlah baris
loops	Berapa kali node dieksekusi

7.3 Indikasi Masalah

1. Seq Scan di tabel besar tanpa index:

Seq Scan on transaksi (cost=0.00..4350.00 rows=1 width=48)

Solusi: Tambah index pada kolom yang difilter.

2. Perbedaan besar antara estimated dan actual rows:

rows=10 vs actual rows=5000

Solusi: Jalankan ANALYZE untuk update statistik.

3. Nested Loop dengan banyak baris:

Nested Loop (actual time=0.05..150.00 rows=5000 loops=1)

-> Seq Scan on a (rows=500)

-> Index Scan on b (rows=10 loops=500)

Solusi: Mungkin butuh Hash Join atau Merge Join — bisa diatur dengan enable_* setting.

7.4 Optimasi Praktis

-- 1. Update statistik agar planner akurat

ANALYZE;

-- 2. Index untuk kolom yang sering di WHERE

CREATE INDEX idx_transaksi_tanggal **ON** transaksi (tanggal);

-- 3. Hindari SELECT * — ambil kolom yang dibutuhkan saja

SELECT id, nama FROM siswa;

-- 4. Gunakan LIMIT jika hanya perlu beberapa baris

SELECT * FROM transaksi **WHERE** id_user = 5 **LIMIT 20;**

-- 5. Partitioning untuk tabel sangat besar (milliaran baris)

```
CREATE TABLE transaksi_2025 PARTITION OF transaksi
FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

7.5 Workflow Diagnosa

1. Query lambat? → EXPLAIN ANALYZE
2. Ada Seq Scan di tabel besar? → Tambah index
3. Estimate meleset? → ANALYZE
4. Masih lambat? → Cek Nested Loop, coba rewrite query
5. Masih lambat? → Cek hardware: RAM, disk I/O, CPU

Rangkuman

Konsep	Inti
JOIN	INNER = irisan, LEFT = semua kiri, RIGHT = semua kanan, FULL = semua
Subquery vs CTE	Subquery untuk sederhana, CTE untuk rekursif/berulang
Window Functions	ROW_NUMBER (unik), RANK (ada lonjakan), DENSE_RANK (tanpa lonjakan)
Transaksi	BEGIN → kerja → COMMIT/ROLLBACK. Jamin atomicity.
Index	B-Tree default. Composite index urutkan kolom paling selektif di depan.
Connection Pool	Kurangi overhead buka-tutup koneksi. pgBouncer = pooler PostgreSQL.
EXPLAIN ANALYZE	Alat utama debug performa. Cari Seq Scan, mismatch rows, Nested Loop

Latihan Soal

1. Buat query menggunakan LEFT JOIN untuk menampilkan semua siswa beserta pembayaran mereka (siswa tanpa pembayaran tetap muncul).
2. Tulis CTE untuk menghitung rata-rata nilai per kelas, lalu gabungkan dengan data siswa.
3. Gunakan ROW_NUMBER() untuk memberi peringkat siswa berdasarkan nilai, per kelas.
4. Simulasikan transfer bank 250.000 dari rekening A ke B dalam satu transaksi. Sertakan ROLLBACK jika saldo A tidak mencukupi.
5. Analisis query berikut dengan EXPLAIN ANALYZE dan sebutkan masalahnya:

```
SELECT * FROM transaksi WHERE YEAR(tanggal) = 2025;
```

Tip: Fungsi YEAR() di WHERE membuat index tidak bisa dipakai. Pakai range: WHERE tanggal >= '2025-01-01' AND tanggal < '2026-01-01'.

Selamat belajar! Database yang dioptimasi dengan baik adalah fondasi aplikasi yang cepat dan andal.